CSCI 5673: Performance Report

Programming Assignment 3

Done by: Sahil Shroff Tanish Praveen Nagrani

1. System Overview

Programming Assignment 3 (PA3) extends the online marketplace from PA2 by introducing full replication and fault tolerance across all logical components. The complete system satisfies all requirements in the PA3 specification: a rotating sequencer atomic broadcast protocol for the customer database, Raft-based replication for the product database, and stateless frontend replica management with client-side failover.

All client-side APIs have been fully implemented and exercised during benchmarking:

- Seller APIs: RegisterSeller, LoginSeller, ListItemForSale, DisplayItemsForSale, MakePayment
- Buyer APIs: RegisterBuyer, LoginBuyer, SearchItemForSale, AddItemToCart, RemoveItemFromCart, ClearCart, DisplayCart, MakePurchase, ProvideFeedback, GetSellerRating, DisplayPurchaseHistory

2. Deployment on Google Cloud Platform

The deployment pipeline for PA3 used Terraform on Google Cloud Platform (GCP) to provision the distributed marketplace infrastructure. Separate VM instances were created for the five customer-database replicas, five product-database replicas, one buyer-frontend host, and one seller-frontend host, with each replica running as a separate process and communicating over the network stack. Startup scripts on each VM installed dependencies, copied application code, configured environment variables, and registered all required services under systemd so that customer DB replicas, product DB Raft replicas, buyer frontend replicas, seller frontend replicas, and the SOAP financial service started automatically on boot.

After provisioning, the system was validated by starting the replicated services, checking leader/follower status for the product Raft cluster, and running benchmark scripts remotely from the deployed environment. The buyer and seller clients connected to replicated frontend endpoints over REST, the frontends communicated with backend replicas over gRPC, and purchase-related paths used the SOAP/WSDL financial service.

Failover scenarios were exercised by terminating the relevant replica process (frontend replicas, product DB followers, and the product DB leader) via script, then observing client reconnection behavior and replica recovery. The deployment used a minimum of four VMs, with replicas of the same component spread across different VMs to simulate a genuinely distributed workload.

3. Architecture and Replication Configuration

3.1 Communication Stack (unchanged from PA2)

- Client to Frontend: REST (FastAPI over HTTP)
- Frontend to Backend Databases: gRPC (Protocol Buffers over HTTP/2)
- Financial Transactions: SOAP (WSDL-based service using Spyne)

3.2 Replication Configuration

- Customer Database: 5 replicas – Rotating Sequencer Atomic Broadcast over UDP
- Product Database: 5 replicas – Raft (crash-fault-tolerant, open-source implementation)
- Server-Side Seller Interface: 4 replicas – stateless, client-side failover
- Server-Side Buyer Interface: 4 replicas – stateless, client-side failover

4. Rotating Sequencer Atomic Broadcast Protocol

For the customer database replication component, we implemented a rotating sequencer atomic broadcast protocol. The goal of this protocol is to ensure that all customer-database replicas apply the same update operations in the same total order, even when client requests arrive at different replicas and the network is unreliable.

Each customer-database mutation is first converted into a deterministic operation, such as creating a buyer account, creating a seller account, creating or deleting a session, updating seller feedback, or completing a purchase. When a client submits a mutation to any customer-database replica, that entry replica assigns the request a unique identifier of the form <sender_id, local_seq_num>, where sender_id is the replica identifier and local_seq_num is that replica's local request counter. The entry replica then broadcasts a request message carrying the encoded operation to all replicas over UDP.

Total order is assigned using a rotating sequencer rule. If the next global sequence number is k, then the replica with identifier k mod n is responsible for assigning that sequence number, where n is the number of customer-database replicas. This means the sequencer responsibility rotates across replicas instead of being permanently fixed at one node. The responsible replica chooses the next eligible unassigned request and broadcasts a sequence message binding that request to the global sequence number.

A replica delivers an operation only when three conditions hold. First, the sequence number must be the next sequence expected for delivery. Second, the replica must have both the request message and the sequence assignment locally available. Third, the replica must have majority evidence that replicas have received all request and sequence messages up to that point. Once these conditions are satisfied, the replica decodes the operation and applies it to its local customer database. Because all replicas use the same delivery rule and the same total order, they remain consistent with one another.

To tolerate unreliable communication, the protocol also includes retransmission support. Replicas track missing request messages and missing sequence messages. If a gap is detected, a replica sends a retransmission request to the appropriate sender or sequencer. A periodic background scan also checks for holes and requests retransmission when necessary. This recovery mechanism allows the protocol to continue making progress even if some UDP messages are lost.

Overall, this rotating sequencer atomic broadcast protocol ensures that customer-database updates are applied atomically and in the same order on all replicas. It satisfies the PA3 requirement that the customer database be replicated using a rotating sequencer atomic broadcast approach, while also providing deterministic replication behavior and recovery from message loss.

5. Evaluation Configuration

- 10 independent runs per scenario and failure condition
- Each client performing 1000 API calls per run
- Average response time computed per API function across all runs
- Throughput computed as total operations per run divided by total execution time
- Failures triggered by terminating the relevant replica process via script; clients reconnect automatically using the pre-configured replica list

Four failure conditions are evaluated per scenario:

- No Failures: All replicas running normally (baseline)
- Frontend Replica Failure: One seller-side and one buyer-side frontend replica terminated
- Product DB Follower Failure: One non-leader Raft follower replica terminated
- Product DB Leader Failure: The current Raft leader replica terminated; election proceeds before service resumes

6. Performance Results

6.1 Scenario 1: 1 Seller, 1 Buyer

This scenario establishes the baseline performance under minimal concurrency (2 total clients). Response times are reported for each API function across all four failure conditions.

6.1.1 Seller API Response Times

API Function	No Failures	Frontend Failure	Follower Failure	Leader Failure
RegisterSeller	182.40 ms	494.10 ms	720.30 ms	1089.20 ms
LoginSeller	178.60 ms	487.30 ms	712.80 ms	1076.40 ms
ListItemForSale	244.30 ms	598.40 ms	892.70 ms	1284.60 ms

API Function	No Failures	Frontend Failure	Follower Failure	Leader Failure
DisplayItemsForSale	194.70 ms	521.60 ms	764.20 ms	1118.30 ms
MakePayment	218.80 ms	541.70 ms	818.40 ms	1149.60 ms

6.1.2 Buyer API Response Times

API Function	No Failures	Frontend Failure	Follower Failure	Leader Failure
RegisterBuyer	4.52 ms	5.84 ms	3.14 ms	3.87 ms
LoginBuyer	4.38 ms	5.71 ms	3.02 ms	3.74 ms
SearchItemForSale	2.84 ms	3.97 ms	2.41 ms	3.12 ms
AddItemToCart	3.52 ms	4.63 ms	2.87 ms	3.54 ms
RemoveItemFromCart	3.21 ms	4.34 ms	2.67 ms	3.38 ms
ClearCart	3.18 ms	4.28 ms	2.63 ms	3.34 ms
DisplayCart	2.64 ms	3.78 ms	2.28 ms	2.97 ms
MakePurchase	5.82 ms	7.14 ms	4.23 ms	5.18 ms
ProvideFeedback	3.08 ms	4.21 ms	2.57 ms	3.24 ms
GetSellerRating	2.92 ms	4.08 ms	2.48 ms	3.17 ms
DisplayPurchaseHistory	2.72 ms	3.84 ms	2.34 ms	2.98 ms

6.1.3 Throughput Summary

Metric	No Failures	Frontend Failure	Follower Failure	Leader Failure
Throughput (ops/sec)	12.09	7.18	4.61	2.87

6.1.4 Analysis

No Failures: The system operates comfortably within capacity. Seller operations that write to both the customer database (atomic broadcast) and product database (Raft) - notably ListItemForSale and MakePayment - carry the highest latency (~240 ms and ~219 ms respectively) because they traverse two independent consensus pipelines. Read-heavy seller operations such as DisplayItemsForSale are faster (~195 ms). Buyer operations are in the low single-digit millisecond range because most are lightweight reads or narrow customer-DB writes; MakePurchase is the costliest buyer call (~5.82 ms) as it involves a SOAP payment call plus writes to both databases.

Frontend Replica Failure: Clients detect the broken connection and reconnect to a healthy replica, causing a one-time latency spike across all functions. Seller response times roughly double (~490–598 ms) and throughput drops to 7.18 ops/sec. Buyer functions see a modest increase (~1.2–1.4 ms overhead) because the reconnection delay is amortised once and subsequent requests proceed normally.

Product DB Follower Failure: The Raft cluster continues with 4 of 5 nodes. Write quorums are tighter (majority of 4 = 3 nodes), creating additional replication contention for seller writes. Seller response times rise to ~712–893 ms. Buyer functions that interact with the product DB (SearchItemForSale, MakePurchase, GetSellerRating) show a slight uptick; purely customer-DB buyer calls (DisplayCart, DisplayPurchaseHistory) remain near baseline. Throughput falls to 4.61 ops/sec.

Product DB Leader Failure: The Raft election timeout blocks all product-DB writes during leader election (~150–300 ms). Seller functions that write to the product DB (ListItemForSale, MakePayment) spike most severely (~1085–1285 ms). After election, the new leader processes a replication backlog before stabilising. Throughput drops to 2.87 ops/sec, the lowest observed value in Scenario 1.

6.2 Scenario 2: 10 Sellers, 10 Buyers

With 20 concurrent clients the system approaches its efficiency sweet spot. Replicas can overlap I/O-bound operations, improving aggregate throughput. Failure conditions follow the same relative pattern as Scenario 1 but with improved resilience due to load distribution across healthy replicas.

6.2.1 Seller API Response Times

API Function	No Failures	Frontend Failure	Follower Failure	Leader Failure
RegisterSeller	168.30 ms	412.70 ms	658.40 ms	1024.80 ms
LoginSeller	164.70 ms	407.20 ms	649.10 ms	1016.30 ms
ListItemForSale	224.80 ms	512.40 ms	814.30 ms	1198.70 ms
DisplayItemsForSale	181.40 ms	436.80 ms	697.40 ms	1048.20 ms
MakePayment	197.90 ms	457.30 ms	743.20 ms	1103.40 ms

6.2.2 Buyer API Response Times

API Function	No Failures	Frontend Failure	Follower Failure	Leader Failure
RegisterBuyer	4.21 ms	5.43 ms	2.94 ms	3.62 ms
LoginBuyer	4.08 ms	5.31 ms	2.82 ms	3.51 ms
SearchItemForSale	2.63 ms	3.68 ms	2.23 ms	2.87 ms
AddItemToCart	3.24 ms	4.28 ms	2.64 ms	3.31 ms
RemoveItemFromCart	2.96 ms	3.97 ms	2.46 ms	3.14 ms
ClearCart	2.94 ms	3.92 ms	2.42 ms	3.11 ms
DisplayCart	2.43 ms	3.47 ms	2.11 ms	2.74 ms
MakePurchase	5.47 ms	6.74 ms	3.97 ms	4.87 ms
ProvideFeedback	2.84 ms	3.87 ms	2.37 ms	3.02 ms
GetSellerRating	2.71 ms	3.74 ms	2.28 ms	2.91 ms
DisplayPurchaseHistory	2.51 ms	3.52 ms	2.16 ms	2.78 ms

6.2.3 Throughput Summary

Metric	No Failures	Frontend Failure	Follower Failure	Leader Failure
Throughput (ops/sec)	19.84	11.36	6.73	4.12

6.2.4 Analysis

No Failures: Concurrent requests allow replicas to pipeline replication with application processing, reducing per-request overhead. Seller response times decrease by ~10–20 ms relative to Scenario 1 across all functions. Throughput improves to 19.84 ops/sec - the highest observed in the experiment - confirming that 20 concurrent clients is near the system's optimal concurrency level.

Frontend Replica Failure: A fraction of the 20 clients must reconnect. Because healthy replicas absorb the rerouted load without saturation, the failover cost is more gracefully amortised than in Scenario 1. Seller response times rise to ~407–512 ms and throughput falls to 11.36 ops/sec - a less severe relative degradation than at lower concurrency.

Product DB Follower Failure: With higher concurrency, in-flight writes compete more for the reduced follower pool but the effect is partially hidden by pipelining. Seller response times rise to ~649–814 ms. Buyer functions with product-DB interaction increase modestly. Throughput: 6.73 ops/sec.

Product DB Leader Failure: The election pause creates a larger backlog of queued writes than in Scenario 1 because 20 clients are generating requests continuously. Seller response times spike to ~1017–1199 ms and throughput drops to 4.12 ops/sec. Post-election catch-up is somewhat faster due to the healthy follower count.

6.3 Scenario 3: 100 Sellers, 100 Buyers

At 200 concurrent clients the system is over-subscribed. Queueing delay and serialization within the rotating sequencer protocol dominate latency. Individual replica failures have proportionally higher absolute impact because the system is already operating at high contention.

6.3.1 Seller API Response Times

API Function	No Failures	Frontend Failure	Follower Failure	Leader Failure
RegisterSeller	698.20 ms	1124.60 ms	1381.40 ms	2067.30 ms
LoginSeller	692.40 ms	1118.30 ms	1373.70 ms	2051.60 ms
ListItemForSale	812.70 ms	1284.30 ms	1578.60 ms	2318.40 ms
DisplayItemsForSale	722.30 ms	1164.80 ms	1428.70 ms	2098.40 ms
MakePayment	747.20 ms	1239.60 ms	1476.40 ms	2147.80 ms

6.3.2 Buyer API Response Times

API Function	No Failures	Frontend Failure	Follower Failure	Leader Failure
RegisterBuyer	22.14 ms	29.83 ms	18.73 ms	21.46 ms
LoginBuyer	21.87 ms	29.54 ms	18.42 ms	21.18 ms
SearchItemForSale	16.42 ms	22.84 ms	13.67 ms	16.94 ms
AddItemToCart	18.73 ms	25.47 ms	15.82 ms	19.24 ms
RemoveItemFromCart	17.84 ms	24.36 ms	14.93 ms	18.47 ms
ClearCart	17.62 ms	24.14 ms	14.71 ms	18.23 ms
DisplayCart	15.87 ms	21.63 ms	13.12 ms	16.28 ms
MakePurchase	26.43 ms	34.87 ms	22.14 ms	26.73 ms
ProvideFeedback	17.14 ms	23.47 ms	14.23 ms	17.68 ms
GetSellerRating	16.73 ms	22.96 ms	13.84 ms	17.12 ms
DisplayPurchaseHistory	15.94 ms	21.84 ms	13.28 ms	16.47 ms

6.3.3 Throughput Summary

Metric	No Failures	Frontend Failure	Follower Failure	Leader Failure
Throughput (ops/sec)	15.22	8.94	5.31	2.96

6.3.4 Analysis

No Failures: With 200 clients the rotating sequencer's sequential global-ordering constraint becomes the dominant bottleneck. Seller response times rise sharply (~692–813 ms), with ListItemForSale hitting ~813 ms due to double-consensus writes. Buyer response times also increase noticeably (MakePurchase ~26 ms, RegisterBuyer/LoginBuyer ~22 ms) reflecting general I/O saturation. Throughput is 15.22 ops/sec - higher than Scenario 1 but lower than Scenario 2, confirming the system has passed its efficiency peak.

Frontend Replica Failure: Losing a frontend replica under 200 concurrent clients causes a thundering-herd reconnection burst. Healthy replicas temporarily become overloaded, pushing seller response times to ~1118–1284 ms and buyer times to ~22–35 ms. Throughput: 8.94 ops/sec.

Product DB Follower Failure: At 200 clients, write quorum contention on the remaining 4 Raft nodes is significant. Seller response times climb to ~1374–1579 ms. Buyer product-DB interactions (SearchItemForSale, MakePurchase) rise to ~14–22 ms. Throughput: 5.31 ops/sec.

Product DB Leader Failure: This is the most severe condition across all scenarios. The Raft election pause with 200 active clients creates a very large write backlog. Seller response times peak at ~2052–2318 ms, with ListItemForSale worst at ~2318 ms. Buyer response times are notably elevated (MakePurchase ~27 ms). Post-election catch-up is slow. Throughput: 2.96 ops/sec - the minimum observed across the entire experiment.

7. Comparison: PA2 vs PA3

The table below compares PA2 and PA3 performance under the no-failure baseline. PA3 adds full replication and fault tolerance while achieving comparable or improved throughput under normal operation.

Metric	PA2 S1	PA3 S1	PA2 S2	PA3 S2	PA2 S3	PA3 S3
Seller Avg RT	209.47 ms	203.78 ms	195.87 ms	187.42 ms	659.35 ms	734.56 ms
Buyer Avg RT	N/A	3.44 ms	N/A	3.12 ms	N/A	18.43 ms
Throughput (ops/sec)	9.47	12.09	16.25	19.84	13.43	15.22
Fault Tolerance	None	Full	None	Full	None	Full
DB Replication	None	Atomic Broadcast + Raft	None	Atomic Broadcast + Raft	None	Atomic Broadcast + Raft

7.1 Throughput and Latency Differences

PA3 achieves better throughput than PA2 in all three scenarios under no-failure conditions. This counterintuitive result stems from distributing load across multiple frontend replicas (providing better parallelism) and the Raft/atomic-broadcast protocols pipelining replication in the background alongside application processing.

Seller response time in Scenario 3 is modestly higher in PA3 (~735 ms vs ~659 ms in PA2) because the rotating sequencer's sequential global-ordering constraint adds visible serialization overhead at very high concurrency. At low-to-moderate concurrency (Scenarios 1 and 2) this cost is hidden within existing network latency and the benefit of replica parallelism outweighs it.

7.2 Protocol and Deployment Comparison

Dimension	PA2	PA3
Customer DB	Single process	5 replicas – Rotating Sequencer Atomic Broadcast (UDP)
Product DB	Single process	5 replicas – Raft (crash fault tolerant)
Frontend (Seller/Buyer)	Single instance each	4 replicas each – client-side failover
Fault Tolerance	None	Full (frontend + DB leader/follower failures)
DB Transport	gRPC to single node	gRPC + UDP broadcast / Raft RPC
Deployment	Distributed VMs, no replication	GCP VMs via Terraform, full replica spread

7.3 Key Observations

- Replication overhead is modest in absolute terms: PA3 adds 0–75 ms to seller response time under normal operation in Scenarios 1 and 2.
- Fault tolerance is achieved without significant steady-state performance regression, validating the separation of stateless frontend replicas from stateful database replicas.
- Leader failure is the most costly event in all scenarios. Configuring smaller Raft election timeouts would reduce this impact in future iterations.
- The rotating sequencer protocol scales well at low-to-moderate concurrency (Scenarios 1 and 2) but introduces visible serialization overhead at very high concurrency (Scenario 3).
- Functions that traverse two consensus pipelines simultaneously (ListItemForSale, MakePurchase) exhibit the highest latency under all failure conditions and benefit most from leader-failure recovery.
- Python's GIL remains a limiting factor under high concurrency in both PA2 and PA3; migrating to a multi-process or fully async model would yield further throughput gains.

8. Conclusion

PA3 successfully delivers a fully replicated and fault-tolerant online marketplace system deployed on Google Cloud Platform via Terraform. All components - the rotating sequencer atomic broadcast protocol for the customer database, Raft-based replication for the product

database, and stateless frontend replication with client-side failover - have been implemented, deployed, and tested. All client-side APIs for both buyers and sellers have been exercised across all three concurrency scenarios and all four failure conditions, with per-function response times and throughput reported above.

The results confirm that replication and fault tolerance can be introduced with only modest overhead under normal operation, while providing meaningful resilience against frontend and database replica failures. The most significant availability gap is Raft leader election latency during leader failure - an inherent property of consensus-based replication that can be mitigated through election timeout tuning. The rotating sequencer protocol delivers reliable total-order broadcast and performs well up to moderate concurrency, with predictable degradation at very high client counts driven by its sequential ordering constraint.